

Python Intermediate–Level Problems

A three-part problem set that combines random data generation, file handling, and data transformation across JSON, YAML, and XML.

Difficulty: Intermediate → Intermediate+ · 3 problems · General constraints on the final page

Topics Covered

- **random** module & random data generation
- **datetime** / **time** operations
- File handling (read & write)
- XML
- YAML
- JSON
- Lists and dictionaries
- Loops and conditional statements
- Basic exception handling
- Data transformation

These problems are designed for students who have **just completed** the concepts above. They require **combining** multiple concepts rather than introducing advanced Python features. Read each problem fully, then build it up function by function.

PROBLEM 1 | Student Performance Data Generator

Problem Description

A college wants to create a **sample student performance dataset** for testing its academic management system.

Write a Python program that generates student records using the **random** module and stores the generated data in a **JSON file**. The program should generate information for a user-specified number of students.

Each student record should contain: student ID, name, age, department, marks in three subjects, total marks, average marks, result (Pass / Fail), and the date of examination.

Requirements

1. Ask the user for the number of students to generate.
2. Generate a unique student ID.
3. Randomly select a student name from a predefined list.
4. Randomly select a department from:
 - Computer Science
 - Information Technology
 - Electronics
 - Mechanical
5. Generate an age between 18 and 25.

6. Generate marks between 0 and 100 for each subject:
 - Python
 - Database
 - Computer Networks
7. Calculate the total marks and the average marks.
8. A student passes only if they score **at least 40 in every subject**.
9. Generate a random examination date within a specified year.
10. Store all student records in **students.json**.
11. After creating the file, read it back and display a summary: total students, number passed, number failed, highest average, and lowest average.

Sample Input

```
Enter number of students: 5
Enter examination year: 2026
```

Sample students.json output

```
[
  {
    "student_id": 1,
    "name": "Arun",
    "age": 21,
    "department": "Computer Science",
    "marks": {
      "python": 82,
      "database": 76,
      "networks": 91
    },
    "total": 249,
    "average": 83.0,
    "result": "Pass",
    "exam_date": "2026-04-18"
  },
  {
    "student_id": 2,
    "name": "Priya",
    "age": 20,
    "department": "Information Technology",
    "marks": {
      "python": 35,
      "database": 72,
      "networks": 68
    },
    "total": 175,
    "average": 58.33,
    "result": "Fail",
    "exam_date": "2026-05-12"
  }
]
```

```
}
]
```

Sample console output

```
Student Performance Summary
-----
Total Students : 5
Passed         : 4
Failed         : 1

Highest Average: 83.00
Lowest Average : 42.67

Student data successfully written to students.json
```

CONCEPTS TESTED

random · random.randint() · random.choice() · datetime · timedelta · JSON · Lists · Dictionaries · File handling · Calculations · Conditional statements

PROBLEM 2 | E-Commerce Order Generator & Configuration Manager

Problem Description

An e-commerce company wants to generate **sample order data** for testing its order-processing system. Create a Python application that generates random orders and stores them in a **JSON file**.

The application should also keep its configuration in a **YAML file** and read settings from it rather than hard-coding them.

Configuration File

Create a file named **config.yaml** containing:

```
store_name: TechStore
minimum_order_amount: 500
maximum_order_amount: 100000
default_currency: INR
allowed_statuses:
  - Pending
  - Shipped
  - Delivered
  - Cancelled
```

The Python program should read these values from the YAML file instead of hard-coding them.

Order Generation Requirements

Ask the user: **How many orders should be generated?** For every order, generate an order ID, customer ID, product, quantity, unit price, total amount, order status, and order date.

Product list

```
Laptop
Mobile Phone
Monitor
Keyboard
Mouse
Headphones
```

Rules

1. Generate a unique order ID.
2. Generate a random customer ID between 1000 and 9999.
3. Select a random product from the list above.
4. Generate a quantity between 1 and 5.
5. Generate a random unit price using `random.uniform()`.
6. Calculate **total_amount = quantity × unit_price**.
7. Select the order status using the values from `config.yaml`.
8. Generate a random order date within the year 2026.
9. Store the generated orders in `orders.json`.
10. Read the JSON file back and display: total number of orders, total sales amount, highest order amount, lowest order amount, number of delivered orders, and number of cancelled orders.

Sample Input

```
Enter number of orders: 5
```

Sample orders.json

```
[
  {
    "order_id": 10001,
    "customer_id": 4821,
    "product": "Laptop",
    "quantity": 2,
    "unit_price": 62500.50,
    "total_amount": 125001.00,
    "status": "Delivered",
    "order_date": "2026-03-18"
  },
  {
    "order_id": 10002,
    "customer_id": 7312,
    "product": "Mouse",
    "quantity": 3,
```

```

        "unit_price": 850.00,
        "total_amount": 2550.00,
        "status": "Pending",
        "order_date": "2026-05-21"
    }
]

```

Sample console output

```

=====
                TechStore Order Report
=====

Total Orders      : 5
Total Sales       : INR 185,450.50
Highest Order     : INR 125,001.00
Lowest Order      : INR 850.00

Delivered Orders  : 2
Cancelled Orders  : 1

Order data saved successfully.

```

Additional Requirement

The program should **not hard-code the order statuses**. For example, avoid this:

```

status = random.choice([
    "Pending",
    "Shipped",
    "Delivered",
    "Cancelled"
])

```

Instead, read the statuses from **config.yaml** and use them for random order generation.

Student note

You may use the **PyYAML** package for YAML processing (`pip install pyyaml`).

CONCEPTS TESTED

random · random.choice() · random.randint() · random.uniform() · datetime · YAML · JSON · File reading · File writing · Nested dictionaries · Lists · Configuration management · Calculations

PROBLEM 3 | Employee Attendance and Monthly Report

Problem Description

A company wants to generate **sample employee attendance data** for testing its HR system. Create a Python application that generates employee attendance records and stores the employee information in **XML**.

The application should then read the XML file, process the attendance information, and generate a **JSON monthly report**.

Employee Data

Generate information for a user-specified number of employees. Each employee should contain an employee ID, name, department, joining date, and attendance records. Use predefined values for names and departments, for example:

```
Departments:
IT
HR
Finance
Sales
Operations
```

Attendance Requirements

Generate attendance for a specified month. For each working day, randomly generate one of **Present**, **Absent**, or **Leave**. Use appropriate probabilities so that **Present** occurs more frequently than **Absent**. For example:

```
Present -> 80%
Absent  -> 10%
Leave    -> 10%
```

You may use **random.choices()** with weights.

XML Output

Store the generated employee data in **employees.xml**. The structure should be similar to:

```
<employees>
  <employee>
    <id>1001</id>
    <name>Arun Kumar</name>
    <department>IT</department>
    <joining_date>2024-06-15</joining_date>

    <attendance>
      <day date="2026-08-01">Present</day>
      <day date="2026-08-02">Present</day>
```

```

        <day date="2026-08-03">Leave</day>
        <day date="2026-08-04">Absent</day>
    </attendance>
</employee>
</employees>

```

Processing Requirements

After creating the XML file:

1. Read the XML file.
2. Extract the employee information.
3. Calculate attendance statistics for every employee.
4. Calculate the following per employee:
 - Total working days
 - Present days
 - Absent days
 - Leave days
 - Attendance percentage
5. Generate a JSON report named **attendance_report.json**.

Use the formula:

$$\text{attendance percentage} = (\text{Present Days} / \text{Total Working Days}) \times 100$$

Sample Input

```

Enter number of employees: 3
Enter year: 2026
Enter month: 8

```

Sample employees.xml

```

<employees>
  <employee>
    <id>1001</id>
    <name>Arun Kumar</name>
    <department>IT</department>
    <joining_date>2024-06-15</joining_date>

    <attendance>
      <day date="2026-08-01">Present</day>
      <day date="2026-08-02">Present</day>
      <day date="2026-08-03">Leave</day>
      <day date="2026-08-04">Present</day>
      <day date="2026-08-05">Absent</day>
    </attendance>
  </employee>
</employees>

```

Sample attendance_report.json

```
[
  {
    "employee_id": 1001,
    "name": "Arun Kumar",
    "department": "IT",
    "total_working_days": 5,
    "present_days": 3,
    "absent_days": 1,
    "leave_days": 1,
    "attendance_percentage": 60.0
  },
  {
    "employee_id": 1002,
    "name": "Priya Sharma",
    "department": "HR",
    "total_working_days": 5,
    "present_days": 4,
    "absent_days": 0,
    "leave_days": 1,
    "attendance_percentage": 80.0
  }
]
```

Sample console output

```
=====
                Employee Attendance Report
=====

Employee ID : 1001
Name       : Arun Kumar
Department : IT

Working Days : 5
Present     : 3
Absent      : 1
Leave        : 1

Attendance  : 60.00%
-----

Employee ID : 1002
Name       : Priya Sharma
Department : HR

Working Days : 5
Present     : 4
Absent      : 0
Leave        : 1
```

Attendance : 80.00%

Attendance report generated successfully.

XML File : employees.xml

JSON File: attendance_report.json

GENERAL CONSTRAINTS | All Problems

1 Use functions

Do not write the entire application inside one block. Break the work into small functions with a single **main()** entry point. Suggested structure:

```
def generate_data():
    pass

def save_data():
    pass

def load_data():
    pass

def generate_report():
    pass

def main():
    pass

if __name__ == "__main__":
    main()
```

2 Use file handling

Students must actually **write to and read from** the files — not just hold the data in memory. The intended flow is:

```
Generate
|
v
Write File
|
v
```

```

Read File
|
v
Process Data
|
v
Generate Report

```

3 Handle invalid input

The application should handle bad input gracefully and show a helpful message instead of crashing — for example:

```
Enter number of students: -5
```

or:

```
Enter number of orders: abc
```

4 Do not hard-code generated values

Values such as the following must be produced using the appropriate Python functionality, not typed in by hand:

```

marks
age
price
quantity
status
attendance
dates

```

Difficulty Progression

Problem	Main File Types	Difficulty	Main Challenge
1. Student Performance Generator	JSON	Intermediate	Random data + calculations + JSON
2. E-Commerce Order Generator	YAML + JSON	Intermediate+	Configuration-driven generation
3. Employee Attendance System	XML + JSON	Intermediate+	XML processing + dates + weighted randomness

These three problems deliberately progress from **random data** to **random data + configuration** to **random data + date calculations + XML-to-JSON transformation**, bridging individual Python concepts to small real-world applications.